REFINED QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

Table of Content

Document Control **3**
FINAL ROUND (Execution, RCA & Hard Evidence) **3**
1. Test Execution & Evidence (Manual) 3
2. Automated Test Coverage Evidence 4
3. Defect Log & Fix Traceability (Root Cause Analysis) 5
4. Known Issues & Deferred Technical Debt 5
5. Live Demo / UAT Risks 6
6. QA Recommendation & Sign-Off 6

Document Control

Field	Detail
System Under Test (SUT)	*[e.g., UMHackathon 2026 - Team YFood]*
Team Repo URL	*[Insert link to your GitHub/GitLab/Bitbucket here]*
Project Board URL	*[Insert link to your Jira/Trello/GitHub Projects board]*
Live Deployment URL	*[Insert link to the live hosted application]*

Objective:

[State the primary goal of your QA process for this specific hackathon project.]

[e.g. The primary objective of this refined QATD is that our team YFood ensured the product producing accurate, hallucination-free dish suggestions. It handles edge cases of inputs and maintains stability. All the GLM outputs and other incidents we encountered are logged and resolved.

FINAL ROUND (Execution, RCA & Hard Evidence)

No evidence = No points.

1. Test Execution & Evidence (Manual)

*Execute the tests drafted in **Part 1**. You must provide a link to visual proof for at least 3 critical tests.*

Test Case ID	Test Type & Mapped Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution (Required for P0/Critical tests)
TC-01	***Happy Case (Entire Flow):*** *End-to-End Flow: Natural Language Orchestration + Booking + HVAC*	*Verify that the system can process a natural language request, perform room booking, trigger pre-cooling, optimize HVAC based on*	*The system should correctly interpret the user's natural language input and successfully complete the*	*Successfully, Order Placed. Payment deducted and receipt is also generated. Vendors receive*	*Passed*	*[Screenshot]*

Test Case ID	Test Type & Mapped Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution (Required for P0/Critical tests)
	Optimization + ESG	*occupancy, and generate accurate ESG data.*	*booking process. It should automatically trigger pre-cooling before the scheduled time and adjust HVAC settings dynamically based on real-time occupancy data. Additionally, the system should accurately record energy usage and generate ESG data without errors or inconsistencies.*	*notification with 1.18s. The rider was assigned and delivered the correct order in time.* *Status: Passed*		
TC-02	***Specific Case (Negative):*** *AI-driven HVAC + Sensor Reliability + ESG Integrity*	*Verify that the system handles incorrect sensor data or AI misprediction without causing unnecessary HVAC or lighting activation or inaccurate ESG reporting.*	*The system should identify inconsistencies or unreliable sensor data and avoid unnecessary activation of HVAC and*	*Payment is blocked, but an error message is not displayed. App Remains stables. The user is not aware of what*	*FAILED*	*[Screenshot]*

Test Case ID	Test Type & Mapped Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution (Required for P0/Critical tests)
			lighting systems. It should fall back to a safe operational state or maintain the current condition until valid data is confirmed. Additionally, the system should flag the anomaly for monitoring and ensure that inaccurate energy consumption is not recorded in the ESG report.	*happened.*		
TC-03	***NFR (Performance):*** *Natural Language Booking + Orchestrator Load Handling*	*Verify that the system can handle multiple concurrent booking requests through the chat interface.*	*The system should identify inconsistencies or unreliable sensor data and avoid unnecessary activation of HVAC and lighting*	*The system should process all booking requests efficiently without system failure or significant delay. It should*	*Average Response time is received as 1128ms. 0% Error Rate.* *Status: Failed*	*[Screenshot URL of Postman Runner results / Terminal output]*

Test Case ID	Test Type & Mapped Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution (Required for P0/Critical tests)
			systems. It should fall back to a safe operational state or maintain the current condition until valid data is confirmed. Additionally, the system should flag the anomaly for monitoring and ensure that inaccurate energy consumption is not recorded in the ESG report.	*correctly assign rooms without duplication and maintain a response time within acceptable limits, ensuring stable system performance under load.*		
TC- 04	***NFR (Performance):*** *AI-driven HVAC Optimization + Sensor Response*	*Verify that the HVAC and lighting systems respond effectively to real-time occupancy changes.*	*The system should detect occupancy changes in real time and automatically activate lighting and adjust HVAC settings accordingly.*	*The system correctly detected occupancy transition in input state and automatically triggered HVAC optimization when occupied.*	*Passed*	*[Screenshot of Prompt & Response + Parsed Recommendation]*

Test Case ID	Test Type & Mapped Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution (Required for P0/Critical tests)
			The response should occur within an acceptable time frame, ensuring both user comfort and energy efficiency without requiring any manual input.	*(Status: PASS) Measured decision times were 5.843e-06s (empty) and 3.406e-06s (occupied), both within the acceptable threshold. However, the lighting system did not automatically activate via the current HVAC agent, as lighting control requires a separate logic implementation to pass.*		
TC-05	***NFR (Performance): ESG Agent + Data Integrity***	*Verify that ESG reports are generated accurately based on validated system data.*	*The system should generate ESG reports that accurately reflect real energy consumption and carbon reduction based on*	*The ESG agent successfully generated ESG reports that accurately matched the raw telemetry data. Cross-verific*	*Pass*	

Test Case ID	Test Type & Mapped Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution (Required for P0/Critical tests)
			verified sensor and system data. The report should not include any fabricated or AI-generated estimations, ensuring full data integrity and reliability.	*ation with telemetry stored in database verifies the generated report.*		

2. Automated Testing Pipeline

2.1. Unit Testing (Core Logic & Functions)

Unit testing is primarily focused on Isolation of the individual components or functions to ensure that these functionalities work perfectly on their own without the need of any external dependencies. Mock your databases or APIs here.

- ***Test Runner Used:*** *[e.g., Jest (Node.js)]*
- ***Targeted Components:*** *[e.g., calculateDeliveryFee(), extractOrderHistory()]*
- ***File(s) Covered:*** *[Link to the test file of your github repo]*

Unit Test ID	*Function/Component Tested*	*Test Scenario*	*Expected Outcome*	*Actual Result*	*Status*

UT-01	*calculateDeliveryFee()*	*Distance over 3.6KM – Verify if the delivery is calculated correctly or not.*	*Returns RM 4.50.*	*Returned RM 9.*	***Failed***

Execution Proof*: [Screenshot of your terminal with the tests conducted or CI/CD proof of Github conducting the tests with results is also applicable]*

2.2. Integration & API Testing (System Modules)

This test is mainly conducting communication (e.g., Frontend to Backend, Backend to Database). Do they integrate correctly?

- **Test Tool Used:** [e.g., Postman automated collections]
- **Targeted Integrations:** [e.g., Backend endpoints connecting to MongoDB, Frontend component fetching data from backend API.]
- **File(s) Covered:** [e.g., Github repo showing Integration Tests or Postman Public Link]

Test ID	Integration / Point Tested	Test Scenario	Expected Outcome	Actual Result	Status
IT-01	[e.g. POST /api/order -> MongoDB]	[e.g. So, this sends a valid order and verify that the database record is	[e.g. Should return HTTP 201. Order should be into Processing State.]	[Actual Execution]	**PASS**

		created and correct status is updated.]			

Execution Proof: [Screenshot of Postman Test Runner Results/ Terminal output of integration tests]

3. Defect Log & Fix Traceability (Root Cause Analysis)

*Log the bugs you **fixed**. Include the exact Git Commit Hash to prove the fix. AI cannot fake your codebase's commit history.*

Bug ID	Bug Description	Steps to Reproduce	Console/Error Log Snippet	Root Cause Analysis (Why did it break?)	Fix Commit Hash / PR Link
DEF-01	*Application crashes when form is submitted with a blank date field.*	*1. Leave the date blank.* *2. Submit.*	Type Error: Cannot read properties of null	*The backend parser attempted to format a null date object. Missing null checks.*	*[Link to commit e.g., github.com/repo/commit/8f4a2b1]*

4. Known Issues & Deferred Technical Debt

It is normal to ship with minor bugs. Document the bugs you found but did not have time to fix. You are highly encouraged to convert these into open GitHub Issues to demonstrate proper defect tracking.

Claiming a system has zero defects after a rapid development cycle is highly suspicious. This section evaluates your transparency and risk awareness. Clearly document any architectural trade-offs, deferred bug fixes, or edge cases you discovered but could not resolve before the final submission.

Note: Proactively documenting minor issues demonstrates engineering maturity and will not negatively impact your score. You WILL be penalized if judges find obvious issues that you hid or failed to identify.

Bug ID	Description & Impact (Bug or Technical Debt)	Reason for Deferral	GitHub Issue Link
DEF-02	***Technical Debt:*** *The admin login is currently hardcoded and doesn't verify against the database.*	*Low priority UI glitch. Focused on fixing database auth logic instead.*	*[Link to open issue e.g., github.com/repo/issues/4]*
DEF-03	***UI Glitch:*** *Dashboard profile image stutters when resizing the browser window to mobile size.*	*Not a core functional blocker. Lack of time to implement responsive CSS.*	*[Link to open issue]*
DEF-04	*[e.g., Data resets if the browser is refreshed because we used local storage instead of a*	*[e.g., Free tier database limits.]*	*[Link to open issue]*

Bug ID	Description & Impact (Bug or Technical Debt)	Reason for Deferral	GitHub Issue Link
	persistent DB.]		

5. Live Demo / UAT Risks

Every prototype has operational limits. Document the fragile parts of your system that judges should know about before they test your application. You are required to identify at least two hard limits or specific operational instructions.

Claiming a prototype has zero risks or boundaries after a rapid development cycle is highly unrealistic. This section evaluates your ability to anticipate failure. Clearly define what your system can and cannot handle so the judging panel evaluates you fairly within those parameters.

Note: Proactively documenting system boundaries protects your demo score and demonstrates proper risk management. You WILL be significantly penalized if you claim, "no risks" and your application crashes or fails unexpectedly during the live demo.

- **Risk 1:** *[e.g., "Do not upload profile pictures larger than 2MB, the free tier server will crash out of memory."]*
- **Risk 2:** *[e.g., "The 'Export to PDF' button takes 10-15 seconds to generate. Please click it only once and wait."]*
- ***Risk 3:*** *[e.g., Browser Compatibility: "This prototype was optimized specifically for Google Chrome. Safari or mobile browsers may experience layout degradation."]*